

Lecture 10

○ Object

- array vs. object
- creation
 - from literal notation
 - as an empty object
 - of nested objects
- methods, i.e.,
function properties

○ Classes

○ Data hiding

getters/setters

○ Inheritance

○ Traversing properties:

- for..of/for..in of for order

○ this, for reusing functions in various contexts

- local vs. global scope
- bindings:
 - implicit
 - explicit
 - new
- Method chaining with this

Object

- consists of *properties*
 - properties are *name: "value"* pairs
 - value can also be a function, the property is then called a method
- is dynamic
 - extra fields can be added after creation
- has a prototype object
 - ES6 added `Object.getPrototypeOf(obj)` to get the prototype instead of using `__proto__` property
- no class until ES6
 - thus, older textbooks do not mention classes
 - Instead, construction with a function

Array vs. object (==associative array)

- An array is indexed using numbers 0 .. N

```
const myArr = ['red', 'blue', 'yellow', 'purple'];
```

- The array can be visualized as follows

key	value
0	' red '
1	' blue '
2	' yellow '
3	' purple '

- Object is similar, but keys can be also strings or Symbols

Creating an object, using literal notation

- Object literal consists of properties as comma-separated name:value pairs, e.g., `firstName: "Mickey"`
- Array is embraced by `[]`, object by `{}`

```
const myObject = {  
  firstName: "Mickey",  
  lastName: "Mouse",  
  isFiction: true,  
  numberOfFriends: 63  
}
```

name	value
firstName	"Mickey"
lastName	"Mouse"
isFiction	true
numberOfFriends	63

Creating an object starting from empty

```
const myObject = {}; // an empty object with curly brackets  
const myObject = new Object(); // ..or with an Object constructor
```

- Next, properties are added to the object on-the-fly
 1. using **dot** notation, or

```
myObject.firstName = "Mickey"; // when value to a non-existing property  
myObject.lastName = "Mouse"; // the property is created  
const name = myObject.firstName + " " + myObject.lastName;
```

2. using **bracket** notation

```
myObject["firstName"] = "Mickey";  
const name = myObject["firstName"];
```

The bracket notation must be used if property name that has a space or a hyphen, or that starts with a number

Nested objects in literal notation

- Nested notation is possible, again note comma separators

```
const triangle = {  
  // Declare properties (each object of two props)  
  p1 : { x : 0, y : 3 },  
  p2 : { x : 1, y : 4 },  
  p3 : { x : 2, y : 5 }  
}  
  
console.log(triangle.p1.y);    // prints 3
```

- The triangle consists of 3 points defined by (x,y) coordinates

Method: lambda as a value of a property

- Object behaviour (*methods*) are created using anonymous functions, such as `fullName()`

```
const person = {  
  firstName : "Mickey",  
  lastName  : "Mouse",  
  isFiction : true,  
  numberOfFriends: 63,  
  fullName : function(){  
    return this.firstName + " " + this.lastName;  
  }  
}  
const p1 = person.fullName();
```

Function expression invoked with ()

- **Without** brackets, the string containing the function expression is assigned as a value

```
let p2 = person.fullName;  
console.log("p2 = " + p2);
```

The following text will be printed on the console:

```
p2 = function () {  
    return this.firstName + " " + this.lastName;  
}  
  
p2 = person.fullName(); //invoked with (), correct!
```

- The brackets invoke a method (or function), and here the return value is assigned to p2
- (function sayHi() {console.log("..remember me?");})();
 - First brackets change f-declaration to f-expression, the latter brackets self-invoke the expression.

this keyword as a reference to a new object

```
let student = {  
  name: "Pasi",  
  sayName: function() {  
    console.log("Hello " + this.name);  
  }  
}
```

- In a method, **this** refers to the owner object.

In a function, to the global object, however, in strict mode, **this** is **undefined**.

- In an event, **this** refers to the element that received the event,
if a handler is not anonymous
- Alone, **this** refers to the global object, but **call()**, **bind()** and **apply()** can bind **this** to
any object. If puzzled, more info, e.g., <https://web.dev/javascript-this/>

Filling in HTML page using JavaScript objects

```
<main class="show">
  <h1>The person:
    <span class="firstname"></span>
    <span class="lastname"></span>
  </h1>
  <p>
    The person is <span class="fiction">#</span>,
    with <span class="friends-nr">#</span> friends.
  </p>
</main>
...
<script src="js/script-with-object-literal.js"></script>
```



JS-12
object example

script-with-object-literal.js

```
const person = { // an object with four properties
  firstName: "Micky",
  lastName: "Mouse",
  isFiction: true,
  numberOfFriends: 63
}
// the object is used to fill in the missing information of the html page template
document.querySelector(".firstname").textContent = person.firstName;
document.querySelector(".lastname").textContent = person.lastName;
document.querySelector(".fiction").textContent = person.isFiction?"fictive":"real";
document.querySelector(".friend-nr").textContent = person.numberOfFriends;
```

Classes in JavaScript since ES6

- A **class** wraps up properties and methods
 - a class can be thought as a 'template'
- Enables easy instantiation of multiple **objects** from the same class
 - objects have same **properties** with different values
 - **methods** may change the values
- Constructor gets automatically called with a **new** operator
 - A constructor method is not the only option, also
 - function constructor / Object.create() factory method

Defining a class

```
class Person {  
    constructor(first,last) {  
        this.firstName = first;  
        this.lastName = last;  
    }  
    fullName() {  
        return this.firstName + " " + this.lastName;  
    }  
    fullNameReversed() {  
        return this.lastName + ", " + this.firstName;  
    }  
} // end of class definition
```

In a constructor, **this** does not yet have a value but it substitutes the new object to be constructed.

Creating objects from a class

```
const p1 = new Person("Billy", "Bragg");  
const p2 = new Person("Rory", "Gallagher");  
  
console.log("Here comes " + p1.firstName);  
console.log("Here comes " + p2.fullName());
```

- p1 and p2 are objects of class Person
 - firstName is a **property** of both objects
 - fullName() is a **method** of both objects

Data hiding

- To prevent the data from being corrupted (intentionally or accidentally)
- To allow only dedicated methods to change the values of properties
 - by providing access to a property via **get** and **set** accessor properties
 - “getters and setters”
 - The underscore (`_name`) is a convention to signify a private variable
- Used in JavaScript library APIs
- Also private instance variables
 - <http://crockford.com/javascript/private.html>

Getters and setters

```
class Person {  
    constructor(nameValue) {  
        this._name = nameValue;  
    }  
    get name() {  
        return this._name;  
    }  
    set name(nameValue) {  
        this._name = nameValue;  
    }  
}
```

```
let p1 = new Person("Harry");  
let nextCustomer = p1.name;
```

JWT

JS-13
getters and setters

getters and setters to access the data

```
get name() {  
    return this._name;  
}
```

- outside the class, it still looks like name is the property
- inside the class, it is implemented as a **get** accessor property

```
set name(name) {  
    this._name = name;  
}
```

- outside the class it still looks like name is the property
- inside the class it is implemented as a **set** method

Hotel class with data hiding

```
class Hotel {  
    constructor(name,address,description,rooms) {  
        this.name = name;  
        this.address = address;  
        this._description = description;  
        this._rooms = rooms;  
        this._bookings = 0;  
    }  
  
    get description(){ return this._description; }  
    get numberOfAvailableRooms(){ return(this._rooms - this._bookings)}  
    set description(description){ this._description = description; }  
}
```

getter can also combine values

```
get numberOfAvailableRooms() {  
    return(this._rooms - this._bookings);  
}
```

- numberOfAvailableRooms is not actually a property, but is used like one outside the class
- inside the class, it is implemented as a **get** method

```
const fineHotel = new Hotel("Fine","Str1","All fine",15);  
let available = fineHotel.numberOfAvailableRooms;
```

Another example of data hiding

```
// we want to prevent this  
fineHotel.description = "Terrible!";
```

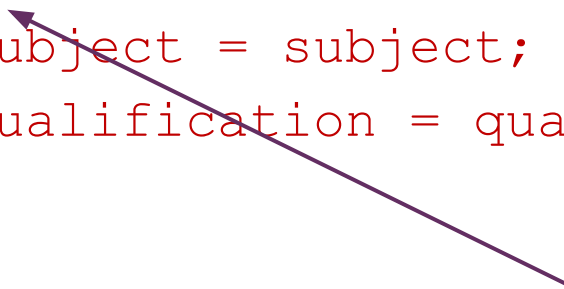
- Disguise the description property by providing no setter

```
console.log(fineHotel.description); //All fine
```

Inheritance by subclassing

- New class (subclass) can be defined based on an existing class

```
class Teacher extends Person {  
    constructor(subject, qualification) {  
        this.subject = subject;  
        this.qualification = qualification;  
    }  
}
```



super()

```
class Person{  
    constructor(first,last){  
        this.firstName=first;  
        this.lastName=last;  
    }  
}
```

- In addition:

- `super()` must be added in the beginning of constructor
 - calls parent's class constructor
- we can use `super()` to pass the super class properties

Subclassing and `super()`

```
class Teacher extends Person {  
    constructor(first, last, subject, qualification){  
        super(first, last);  
        // subject and qualification  
        this.subject = subject;  
        this.qualification = qualification;  
    }  
}  
  
const teacher = new Teacher ('Riitta','Rapeli','mathematics','professor');
```

Prototypal (JS) vs. classical inheritance (e.g. Java, C++)

Building blocks	Object: bag of properties, key-value pairs	Interfaces, abstract classes, classes instantiated as objects
Construction/ Instantiation	- From string: <code>{}</code> //object, <code>[]</code> //array <code>const o={}</code> <code>o.prototype === Object</code> Object.create(obj) - Object constructors (function Person(first,last) - From class, let <code>p = new Person("Pia","N")</code>; <code>class Person(first,last){constructor(first,last){}}</code>	Object is constructed based on a class, e.g. <code>Person p = new Person();</code>
Inheritance	- function Teacher(first, last){Person.call(this, first,last)} - class Teacher extends Person (+ super() in constructor)	class O extends superO implements Interface
Augmenting/ Decorating	<code>const getName = function(){return `\${this.first} \${this.last}`;}</code> <code>Person.prototype.name = getName //all Persons</code> <code>p.name = getName //only p</code>	New methods introduced by changing interfaces/classes
Visibility	getters/setters, Object.defineProperty() , Object.getOwnPropertyNames(o)	private, protected, public (friends); reflection
Destruction	Garbage collection (reference counted)	Garbage collection, setting to null

Security vulnerability: prototype pollution

Array of objects

- a common means of providing a collection of objects

```
let register = new Array();  
let numberOfStudents = 0;
```

```
register[numberOfStudents++] = new Person("Bob", "Scott");  
register[numberOfStudents++] = new Person("Joe", "Jump");
```

```
console.log(register.length);           // displays 2  
console.log(register[1].lastName);      // displays "Jump"
```


for .. of for iterables such as arrays keeps the order

new in ES6

```
let allKeys = "", allValues = "";  
let mylist = new Array("Chinese", "English", "French");  
mylist.newField1 = "Something";
```

```
for (let key in mylist) { // ok, but does not guarantee  
  allKeys += key + " "; // the access order (!)  
  allValues += mylist[key] + " ";  
}
```

```
for (let key of mylist) { // access in the indexing order  
  allKeys += key + " ";  
  allValues += mylist[key] + " ";  
}
```

```
// allKeys contains "0 1 2 newField1"
```

```
// allValues contains "Chinese English French Something"
```

for ..in for objects

```
for (let variable in object) {  
    statements;  
}
```

- Special syntax to iterate through all the properties in object
 - **variable** takes a name of each property in object
 - no guarantee of the order in which the properties are accessed
- Can be used to iterate all the elements in an Array object, but again, order not guaranteed

this and global scope

- If a function is declared in the global scope, `this` refers to the window:

```
function sayHello(){  
    console.log("Hello " + this.sname);  
}  
sayHello();    // Hello undefined
```

- No global variable `sname`, thus prints undefined

- If global `sname` is defined, its value will be used:

```
const sname = "Boris";  
function sayHello(){  
    console.log("Hello " + this.sname);  
}  
sayHello();    // Hello Boris
```

// this not necessary,
// as the global object
// is used anyway

this and local scope

Binding to local scope applies only to a conventional function (named or anonymous)

```
function sayHelloforAll(){console.log("Hello "+ this.sname)}
```

```
var s2 = {  
  sname:"Mikko",  
  sayHello: sayHelloforAll  
}
```

```
var s3 = {  
  sname:"Anu",  
  sayHello: sayHelloforAll  
}
```

```
s2.sayHello(); // Hello Mikko  
s3.sayHello(); // Hello Anu
```

the value of `this` depends on which object has called the function. In the example, it is either `s2` or `s3`

this and implicit binding

- In a conventional function, `this` refers to the global object
 - `window` in frontend and `global` in backend code (node.js)
 - if a function is set as a method, `this` refers to an owner object (`s2/s3` in the prev example)
- `eval()`
 - evaluates a given string as JavaScript code and executes it
 - `this` stays the same as `eval`'s context
- however, `this` can be set deliberately to a different value with explicit binding

this and explicit binding

- Predefined methods of Function objects(ES5) `call()`, `apply()` and `bind()`
 - allow setting `this` explicitly
`sayHelloforAll.call(s2); //Hello Mikko`
`sayHelloforAll.call(s3); //Hello Anu`
- Function is an object in JavaScript
 - it has its own methods through `Function.prototype` (i.e., `call()`, `apply()` and `bind()`)

this set explicitly with call()

```
function sayHelloforAll(greeting){  
  console.log(greeting + " " + this.sname);  
}
```

```
var s2 = {  
  sname: "Mikko"  
}
```

```
var s3 = {  
  sname: "Anu"  
}
```

```
sayHelloforAll.call(s2, "Hi s2");
```

```
sayHelloforAll.call(s3, "(to s3), Hi");
```

the first parameter
set as this

remaining parameters
are passed to the
function
sayHelloforAll()

this and new binding: conventional constructor

- Behind the scenes, the conventional constructor
 - creates a new `this` object and returns it

```
function Student(sname,age,homeTown) {  
    this.sname = sname;  
    this.age = age;  
    this.homeTown = homeTown;  
}  
const s2 = new Student("Lena", 23, "Tampere");
```


Method chaining as a design pattern exploiting `this`

- Important to recognise what is happening:

```
let s1 = new Student()  
    .setFirstName('Bob')  
    .setLastName('Harris')  
    .setCourseName('Interactive Media')  
    .setEntryYear(2017)  
    .print();
```

- only one statement chaining all the methods!

Normal no-chain vs. method chain

```
class Student1{
  constructor(){
    this.firstName = "first not set";
    this.lastName = "second not set";
  }
  setFirstName(value){
    this.firstName = value;
  }
  setLastName(value){
    this.lastName = value;
  }
}
// then call each method separately
var s1 = new Student();
s1.setFirstName('Bob');
s1.setLastName('Harris');
```

```
class Student2{
  constructor(){
    this.firstName = "first not set";
    this.lastName = "second not set";
  }
  setFirstName(value){
    this.firstName = value;
    return this;
  }
  setLastName(value){
    this.lastName = value;
    return this;
  }
}
let s1 = new Student()
    .setFirstName('Bob')
    .setLastName('Harris');
```

Kickoff meeting 07.04 16-18

Team should

- Introduce themselves
- Start with JS2: kickoff and game-based learning quizzes
- Agree on next meetings and continuation in general
 - If a member is missing, **please inform the course personnel**
- The roles in the teams are: PM, architect, developer
 - Everyone is coding but
 - PM should be more proactive in, e.g., arranging meetings and good communication
 - architect might know more about JS(based on performance in previous exercises), in this case sharing his/her knowledge
 - Submit code exercises together